ТЕХНИЧЕСКИЙ ДОКУМЕНТ · АРХИТЕКТУРА

Солнечный зайчик
Башня последнего света

Устройство 3D-игры на Three.js: модули, модель мира, игровой цикл и графические системы

Документ описывает фактическую реализацию проекта: параметрическую модель башни, разделение игровой логики и рендера, системы света, угроз, витражей и визуальных эффектов, а также тестирование, сборку и точки расширения.

Версия проекта	1.0.0
Стек	JavaScript (ES-модули), Three.js 0.185, Vite 7.2, node:test
Дата документа	10 ноября 2026
Объём кода	15 модулей в `src/`, 13 файлов тестов

Содержание

1 Резюме проекта
2 Стек и структура репозитория
3 Архитектурные принципы
4 Карта модулей
5 Граф зависимостей
6 Запуск и жизненный цикл
7 Модель мира
8 Состояние игры
9 Игровой цикл
10 Ввод и управление
11 Прыжок и коллизии
12 Свет как здоровье
13 Угрозы
14 Окна, витражи и свет
15 Теневые звери
16 Визуальные эффекты
17 Камера
18 Освещение и шейдеры
19 Интерфейс
20 Звук
21 Производительность
22 Тестирование
23 Сборка и развёртывание
24 Точки расширения
25 Расхождения и технический долг

О документе

Все числа, формулы и имена функций приведены по текущему состоянию кода в каталоге `src/`. Документ описывает именно реализацию, а не замысел: если README и код расходятся, приоритет отдан коду, а расхождение зафиксировано в разделе 25.

1. Резюме проекта

«Солнечный зайчик — Башня последнего света» — законченная браузерная 3D-игра на Three.js. Игрок управляет солнечным зайчиком, который спускается по спирали магической башни из светлого верхнего этажа к тёмному нижнему, чтобы донести последний свет заката до кристалла.

Параметр	Значение
Жанр	3D-аркада с прыжками, уклонением и сбором предметов
Длительность сессии	3–5 минут, один уровень из 12 этажей
Маршрут	168 секций (0…167), шесть витков спирали
Ресурс	«Свет» 0…100, он же здоровье и яркость модели
Графика	Полностью процедурная: геометрия, текстуры и эффекты создаются кодом
Ассеты	Внешних моделей, изображений, шрифтов и CDN нет
Звук	Синтез Web Audio API после жеста пользователя
Сборка	Vite, статический `dist/`, относительный `base: './'`
Требования	Node.js 22.12+, браузер с WebGL 2 и аппаратным ускорением

Архитектурно проект — это *одностраничное приложение без UI-фреймворка*: интерфейс свёрстан обычным HTML/CSS, а вся трёхмерная сцена и игровая симуляция живут в одном модуле композиции `main.js`, который использует набор специализированных модулей. Ключевая особенность — жёсткое разделение «чистой» игровой математики (без Three.js) и слоя представления.

2. Стек и структура репозитория

```
Sunny/
├── index.html              разметка интерфейса, canvas, HUD, модальные окна
├── package.json            three, vite, скрипты dev/build/preview/test
├── vite.config.js          base: './' для GitHub Pages
├── README.md               описание игры и правил
├── .github/workflows/
│   └── deploy.yml          test → build → публикация dist на GitHub Pages
├── src/
│   ├── main.js             композиция сцены, состояние, игровой цикл (≈330 строк)
│   ├── game.js             чистая математика: маршрут, прыжок, свет, камера-якорь
│   ├── camera.js           сглаживание и поза камеры
│   ├── threats.js          реестр угроз: провалы и подвижные полки
│   ├── window-light.js     лучи из окон, проекция витражей, зоны восстановления
│   ├── stained-glass.js    процедурные текстуры витражей и «розы»
│   ├── shadow-beast.js     четыре вида теневых зверей и их смерть
│   ├── shadow-effects.js   экранный эффект теневого урона и тряска камеры
│   ├── pickup-effects.js   сбор блика: сжатие, вспышка, искры
│   ├── ground-shadows.js   тёмные пятна на полках и поднимающийся дым
│   ├── tower-mist.js       объёмный туман и проход глубины
│   ├── tower-depth.js      затемнение архитектуры по высоте (инъекция в шейдер)
│   ├── vitral-glow.js      цветное свечение зайчика в витраже
│   ├── rabbit-animation.js процедурная анимация тела, ушей, лап и хвоста
│   └── style.css           оформление интерфейса и сенсорных кнопок
└── tests/                  13 файлов, 28 тестов на node:test
```

Зависимостей всего две: `three` (рантайм) и `vite` (сборка). Тестовая инфраструктура встроена в Node.js и не требует отдельных пакетов.

3. Архитектурные принципы

3.1. Логика отделена от представления

Модули `game.js`, `camera.js` и `threats.js` не импортируют Three.js. Они работают с числами и простыми объектами. Это даёт три выгоды:

- симуляцию можно запустить в Node.js без WebGL и браузера — на этом построены тесты;
- правила игры не зависят от частоты кадров и разрешения;
- визуальные модули можно менять, не рискуя сломать баланс.

Three.js появляется только там, где есть геометрия, материалы, спрайты и шейдеры. `main.js` — единственное место, где эти два мира соединяются.

3.2. Мир параметризован, а не разложен по координатам

Позиция игрока задаётся не вектором, а парой чисел: `s` — расстояние вдоль спирали (0…167) и `lane` — смещение по радиусу (−1,8…1,8). Все координаты выводятся функцией `route(s, lane)`. Никаких массивов заранее расставленных объектов нет.

Следствие: проверка коллизий сводится к сравнению скаляров (`|s − h.s| < 0.65`), а расстановка предметов — к арифметике по индексу секции. Это резко уменьшает объём кода и делает поведение воспроизводимым.

3.3. Сущности — записи в массивах, а не классы

Блики, тени, склянки, книги, звери и угрозы хранятся в обычных массивах объектов-записей: `{ s, lane, o, taken }`. Обновление — один проход циклом в `update(dt)`. Классы применяются только там, где есть собственное состояние и жизненный цикл: `RabbitAnimation`, `PickupEffects`, `ShadowEffects`, `TowerMist`, `VitralGlow`, `GroundShadowSmoke`.

Такой подход дешевле по памяти и проще для отладки: любую сущность видно в консоли как данные.

3.4. Детерминизм и процедурная генерация

Случайность нужна только для декора: книги на полках и длина полок. Используется линейный конгруэнтный генератор с фиксированным зерном 42, поэтому башня выглядит одинаково при каждом запуске. Длина конкретной полки считается хешем от номера секции (`shelfInnerRadius`) — это чистая функция, её можно проверить тестом.

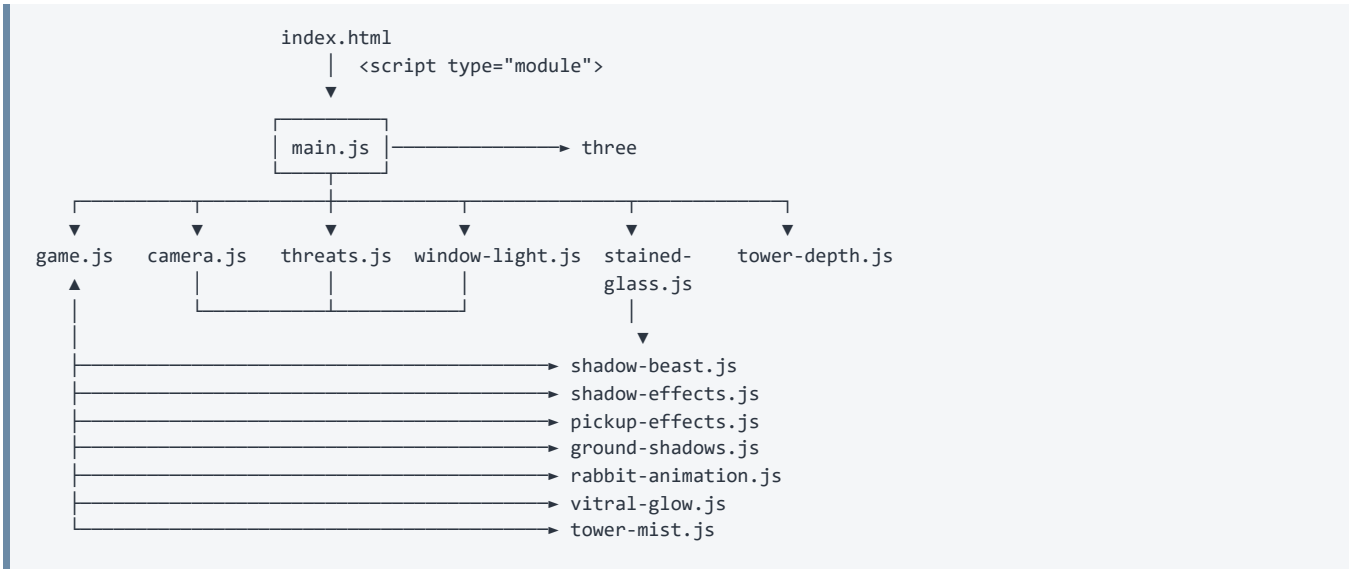
3.5. Бюджет кадра важнее «красоты кода»

В горячем пути избегаются создания материалов, геометрий и спрайтов: искры блика — один `InstancedMesh`, книги декора — шесть instanced-вызовов, дым зверей — заранее созданный пул из 18 спрайтов, туман — 96 инстансов одной плоскости. Часть компромиссов (короткоживущие объекты, возвращаемые `route()`) осознанно оставлена ради читаемости; они перечислены в разделе 21.

4. Карта модулей

Модуль	Ответственность	Ключевые экспорты
`main.js`	Композиция сцены, материалы, расстановка сущностей, состояние игры, ввод, HUD, игровой цикл и рендер	— (точка входа)
`game.js`	Константы мира, маршрут, разрывы, этажи, прыжок, расход света, поворот, якорь камеры	`route`, `advanceJump`, `lightStep`, `cameraAnchor`, `END`
`camera.js`	Критически задемпфированное сглаживание якоря и расчёт позы камеры	`smoothCameraAnchor`, `cameraPose`
`threats.js`	Реестр угроз, цикл движения полки, проверка опоры под игроком	`createThreats`, `platformMotion`, `unsupportedThreat`
`window-light.js`	Лучи из каждой грани окна, проекция витража на полку, зоны восстановления света	`createWindowLight`, `paneBeam`
`stained-glass.js`	Процедурные canvas-текстуры: 4 узких и 4 широких мозаики, круглое окно-роза	`createGlassTextures`, `glassLayout`
`shadow-beast.js`	Сборка зверей из эллипсоидов, анимация, умерщвление с растворением	`createShadowBeast`, `killShadowBeast`, `animateBeastDeath`
`shadow-effects.js`	Реакция на теневой урон: пульс, экранное затемнение, тряска камеры	`ShadowEffects`
`pickup-effects.js`	Двухфазный эффект сбора блика с освобождением ресурсов	`PickupEffects`
`ground-shadows.js`	Шейдерное пятно тени и восходящий дым над опасными зонами	`softShadowMaterial`, `GroundShadowSmoke`
`tower-mist.js`	Объёмный туман на инстансах, шум FBM, проход глубины для контакта с геометрией	`TowerMist`
`tower-depth.js`	Затемнение и «синева» архитектуры по мировой высоте через инъекцию в шейдер	`applyTowerDepth`
`vitral-glow.js`	Цветные лучи вокруг зайчика, пока он стоит в витраже	`VitralGlow`
`rabbit-animation.js`	Процедурная походка, прыжок, приземление, наклон и поворот	`RabbitAnimation`
`style.css`	Оформление панелей, HUD, модальных окон, сенсорных кнопок	—

5. Граф зависимостей



Стрелка «►» означает импорт. `game.js` — корень графа: его импортируют `main.js`, `camera.js`, `threats.js` и `window-light.js`. Сам он не импортирует ничего, кроме стандартного JavaScript. Это и есть граница «симуляция / рендер».

6. Запуск и жизненный цикл

6.1. Фаза загрузки

1. `index.html` создаёт `<canvas id="world">`, HUD, интро-панель, модальное окно, тост и сенсорные кнопки.
2. `main.js` создаёт сцену, фон и экспоненциальный туман.
3. Инициализируется `WebGLRenderer` в `try/catch`: при отсутствии WebGL интро-панель заменяется сообщением «Нужен WebGL».
4. Строится статическая архитектура: цилиндр башни, 16 колонн, 12 ярусов, 192 книжных шкафа, окна, ловец солнца, кристалл, пыль.
5. Создаются процедурные текстуры витражей и материалы.
6. Генерируются сущности по индексам секций: полки, разрывы, блики, тени, склянки, звери, книги.
7. Книги декора пакетируются в `InstancedMesh` (6 вызовов отрисовки вместо сотен).
8. Создаются системы: лучи окон, туман, эффекты, анимация зайчика, HUD-обработчики.
9. Запускается `requestAnimationFrame(frame)`.

6.2. Фаза игры

Игра стартует не автоматически: нажатие «Разбудить свет» вызывает `start()`, которая выполняет `reset()` и переводит автомат в `playing`. До этого камера медленно облетает зайчика в режиме `intro`.

6.3. Перезапуск

`reset()` — единая точка возврата в исходное состояние. Она сбрасывает:

- все скалярные поля игрока (`s`, `lane`, `height`, `light` и т.д.);
- угрозы, витражное свечение, анимацию, эффекты, счётчик бликов;
- видимость и масштаб бликов, кулдауны склянок, состояние зверей;
- освещение, цвет фона, туман и эмиссию стекла.

Благодаря этому «Начать заново» после победы, поражения или паузы не требует перезагрузки страницы и не оставляет «мусора» в сцене.

7. Модель мира

7.1. Спиральный маршрут

Вся геометрия уровня выводится из одной функции:

```
θ = s · π/14    r = 12 + lane
x = r · cos θ    z = r · sin θ    y = 100 − 0,5 · s
```

Здесь `s ∈ [0, 167]` — номер секции, `lane ∈ [-1,8; 1,8]` — поперечное смещение. Шаг спирали — 14 секций на оборот (≈ 12,86°), поэтому весь маршрут — почти шесть полных витков. Высота линейно падает со 100 до 16,5 единиц.

Константа	Значение	Смысл
`END`	167	Последняя секция маршрута; кристалл стоит на 168-й
`FLOOR_COUNT`	12	Количество этажей
`ROUTE_ANGLE`	π/14	Угол одной секции
`SHELF_HALF_ANGLE`	0,133 рад	Половина угловой ширины полки (≈ 7,6°)
`SHELF_OUTER_RADIUS`	14,7	Внешний радиус полки
`SHELF_INNER_RADIUS`	9,3	Базовый внутренний радиус
`LANE_LIMIT`	1,8	Предел поперечного смещения
`JUMP_SPEED`	9	Начальная скорость прыжка
`GRAVITY`	16	Ускорение свободного падения

7.2. Полки и разрывы

Каждая полка — это `ExtrudeGeometry` из плоской фигуры: внешняя дуга радиусом 14,7 и внутренняя дуга радиусом `shelfInnerRadius(s)`, между углами ±0,133 рад. Толщина 0,7; сверху добавляется тонкий бортик-сегмент торуса. Внутренний радиус не постоянен: хеш от номера секции даёт отклонение до ⅛

базовой длины, поэтому полки выглядят «живыми», но остаются проходимыми (тест `shelf-length.test.js`).

Разрывы заданы списком `gaps = [19, 38, 59, 81, 103, 126, 147]`. Полуширина разрыва вычисляется из геометрии, а не подбирается вручную:

```
GAP_HALF_WIDTH = 1 − SHELF_HALF_ANGLE / ROUTE_ANGLE ≈ 0,407
```

То есть разрыв занимает ровно то угловое пространство, где соседние полки не доходят друг до друга. Полки на этих секциях не создаются вовсе, а `isGap(s)` используется и для коллизий, и для расстановки объектов.

7.3. Этажи и угасание света

```
floorAt(s) = 12 − min(11, ⌊clamp(s, 0, END) / 14⌋)
depthLight(s) = (1 − s / END)^1,25
```

`floorAt` отображает секцию в номер этажа (12 — верх, 1 — низ). `depthLight` — нормированная «сила заката»: 1 наверху, 0 у кристалла. Она управляет яркостью полусферического света, солнца, тёплой точки, тумана, фона и эмиссии витражей. Это единственный источник «глобального времени суток» в игре.

7.4. Архитектура башни

- Внешняя оболочка — открытый цилиндр радиусом 19 и высотой 132, материал с `BackSide`: игрок видит стену изнутри, а сердце башни остаётся пустым.
- 16 колонн радиусом 18,5 связывают ярусы.
- 12 ярусов: торус-обод и 16 книжных шкафов на каждом. Шкаф — группа из корпуса и шести книг.
- Высокие окна на верхнем ярусе (8 окон по 18 стеклянных панелей) и пояс окон ниже (6 уровней по 8 окон, по 4 панели). Всего 336 панелей.
- Ловец солнца, кристалл с точечным светом, 650 частиц пыли, финальное кольцо.
- Книги декора после построения сцены собираются в `InstancedMesh` по материалам.

8. Состояние игры

Состояние — это набор переменных модуля `main.js` плюс одна строковая переменная `state`. Отдельного хранилища нет: замыкание модуля и есть модель.

Поле	Начало	Назначение
`state`	`'intro'`	Фаза: `intro`, `playing`, `paused`, `won`, `lost`
`s`	0	Позиция вдоль спирали
`lane`	0	Поперечное смещение
`height`, `velocity`	0, 0	Вертикальная координата и скорость
`light`	100	Свет = здоровье
`collected`	0	Собрано бликов
`invert`, `slow`	0, 0	Таймеры эффектов склянок
`lastSafe`	0	Последняя безопасная секция для «спасения»
`facing`, `facingTarget`	−0,5	Текущий и целевой угол поворота
`currentVitralZone`	`null`	Зона витража под игроком
`time`	0	Игровое время; замирает на паузе
`keys`, `jumpQueued`	пусто, false	Ввод



Переходы выполняются только в трёх функциях: `start()`, `pause()`, `finish(won)`. Это делает поведение предсказуемым: ни один игровой обработчик не меняет `state` напрямую.

9. Игровой цикл

```
function frame(now) {
  requestAnimationFrame(frame);
  const dt = Math.min((now – previous) / 1000, 0.04);
  previous = now;
  visualTime += dt;
  if (state === 'playing') update(dt);
  towerMist.update(...);
  ...
  renderer.render(scene, camera);
}
```

Два времени разделены намеренно: `time` — игровое, оно замирает на паузе; `visualTime` — анимационное, продолжает идти в интро и финале. Шаг `dt` ограничен 0,04 с, чтобы после переключения вкладки или подтормаживания физика не «прыгала» на полсекунды вперёд.

9.1. Порядок шагов `update(dt)`

1. Запоминается высота стопы до движения — она нужна для проверки «прыжка на голову» зверю.
2. Увеличивается `time`, уменьшаются таймеры `invert` и `slow`.
3. Обновляются угрозы (движение полок, предупреждения).
4. Считывается ввод: `forward` (по спирали) и `radial` (к краю/вглубь).
5. Применяется движение с учётом инверсии и замедления, обновляются `s`, `lane`, скорость и целевой угол поворота.
6. Обрабатывается прыжок: постановка в очередь → `advanceJump`.
7. Проверяются тени на полу; при отсутствии опоры — провал и «спасательный» подскок.
8. Собираются блики, срабатывают склянки.
9. Обновляются звери: преследование, анимация, проверка прыжка на голову, урон.
10. Обновляются падающие книги.
11. Определяется зона витража и применяется `lightStep`.
12. Обновляются HUD, эффекты, проверяется финал (`light ≤ 0` или `s ≥ END − 0,1`).

Порядок фиксирован: сначала угрозы и движение, затем проверка опоры, затем сбор и урон, затем расход света и только в конце — условия победы/поражения. Это исключает двойное начисление урона и «победу после смерти» в одном кадре.

10. Ввод и управление

- Клавиатура: `A`/`D` и стрелки — вдоль спирали; `W`/`S` — к внешнему/внутреннему краю; `Space` — прыжок; `Esc` — пауза.
- Разрешённый список кодов проверяется до `preventDefault()`, чтобы не блокировать остальные клавиши браузера.
- Прыжок не выполняется сразу в обработчике: он ставится в очередь `jumpQueued` и обрабатывается в `update`. Так прыжок не зависит от частоты событий.
- Сенсорные кнопки используют `pointerdown` / `pointerup` и `setPointerCapture`: можно удерживать движение и одновременно нажимать прыжок.
- При потере фокуса окна или скрытии вкладки игра ставится на паузу и очищает нажатия — иначе зайчик «уезжал» бы с зажатой клавишей.
- Инверсия управления (`invert`) не меняет считывание клавиш, а умножает направление на −1 — поэтому визуальная подсветка HUD и физика остаются согласованными.

11. Прыжок и коллизии

11.1. Вертикальная физика

```
h ← h + v·dt − ½·g·dt²    v ← v − g·dt    (g = 16)
при h ≤ 0: h = 0, v = 0
```

Начальная скорость 9 даёт максимальную высоту $9^2/(2 \cdot 16) = 2{,}53$ единицы и время полёта 1,125 с. При горизонтальной скорости 1,45 секции/с за прыжок проходится около 1,63 секции — почти вдвое больше ширины разрыва (0,815 секции). Запас нужен для прыжков «с края» и на низком FPS.

Тест `jump.test.js` прогоняет все семь разрывов в обе стороны на 25, 30, 60, 120 и 144 FPS в трёх режимах: прыжок с края, ранний прыжок и «спасательный» прыжок после провала. Ни один вариант не должен привести к падению.

11.2. Провал и последняя безопасная позиция

Если игрок стоит на полу (`height = 0`, `velocity ≤ 0`) и под ним нет опоры (`unsupportedThreat`), он теряет 14 света, возвращается в `lastSafe` и получает небольшой подскок (`velocity = JUMP_SPEED`, `height = 0,2`). `lastSafe` обновляется только на твёрдой полке, которая не является подвижной.

11.3. Прыжок на зверя

```
canStomp(prevFoot, foot, prevTop, top, v, overlaps) = overlaps && v < 0 && prevFoot ≥ prevTop − 0,12 && foot ≤ top + 0,12
```

Условие требует одновременного выполнения четырёх фактов: горизонтальное перекрытие, движение вниз, стопа была не ниже верхней точки зверя в прошлом кадре и оказалась не выше неё в текущем. Такой вариант устойчив к проскоку на низком FPS и не срабатывает при боковом касании.

12. Свет как здоровье

Единственный ресурс игры — «свет» (0…100). Он же здоровье, он же яркость модели зайчика.

Событие	Изменение	Где в коде
Пассивное угасание	−12 в секунду	`PASSIVE_LIGHT_DRAIN`
Теневая зона / зверь / книга	−48 в секунду	`lightStep(..., { shadow: true })`
Витраж	+24 в секунду	`VITRAL_LIGHT_REGEN`
Золотой блик	+22 мгновенно	`update`, сбор блика
Провал / полка ушла	−14 мгновенно	`update`, `missingSupport`
Прыжок на зверя	без урона, зверь рассеивается	`canStomp` → `killShadowBeast`

```
lightStep(L, dt, {shadow, vitral}) = clamp(L + ((vitral ? 24 : 0) − (shadow ? 48 : 12)) · dt, 0, 100)
```

Важная деталь баланса: витраж не даёт иммунитета. Пока игрок стоит в цветном пятне, восстановление (+24/с) и пассивная потеря (−12/с) дают чистый прирост +12/с; при этом теневой урон (−48/с) продолжает действовать, и в тени витраж уже не спасает. Это проверяет `game.test.js`.

12.1. Яркость модели

```
t = clamp(L/100, 0, 1)    brightness = (t³ / (t³ + (1−t)³))^1,4
```

Кубическое отношение даёт резкое падение яркости ниже половины шкалы: при 50 % brightness ≈ 0,33, при 25 % — менее 0,02. Значение подаётся в шейдер зайчика как uniform `rabbitHPFade` и умножает цвет после тумана, а также управляет цветом глаз, блика и цветного свечения. Так игрок видит запас света без взгляда на HUD.

13. Угрозы

Угрозы описаны декларативно в `threats.js`. Есть два типа:

Тип	Поведение
`hole`	Статический разрыв в маршруте. Опора отсутствует, если `|s − hole.s| < GAP_HALF_WIDTH`.
`moving-platform`	Полка, которая периодически уходит внутрь башни на 2,1 единицы и возвращается.

13.1. Реестр

`createThreats(holes, END, reserved)` превращает список разрывов в записи-угрозы и добавляет подвижные полки с шагом 18 секций, начиная с 12-й. Кандидат отбрасывается, если он зарезервирован под другой объект или стоит ближе трёх секций к разрыву. Резервирование задаётся множеством `reservedThreatShelves` — это те же формулы, по которым расставляются блики, тени, звери и книги. Так системы не накладываются друг на друга.

13.2. Цикл подвижной полки

```
t = (time − 5) mod 5    offset = −2,1 · amount(t)
0…0,45 с — плавный уход (smoothstep)  0,45…0,95 с — удержание
0,95…1,45 с — возврат    t ≥ 4,25 с — предупреждение
```

Полка не наносит урон в воздухе. Проверка опоры выполняется в локальных координатах полки: точка игрока смещается на `offset`, затем проверяется, что её радиус лежит между внутренним и внешним радиусом полки, а угол — внутри половины угловой ширины. Если внешний край ушёл, игрок проваливается; помогает прыжок или переход к внутреннему краю (тест `threats.test.js`).

13.3. Индикация

У подвижной полки есть собственный меш-индикатор — копия бортика с оранжевым эмиссивным материалом. Он появляется за 0,75 с до движения и пульсирует. Дополнительно за две секции до полки показывается тост «Полка сдвигается!». Предупреждение сбрасывается, когда движение закончилось, чтобы его нельзя было пропустить.

14. Окна, витражи и свет

14.1. Процедурные текстуры

`stained-glass.js` рисует мозаики на `<canvas>`. Плотность текселей постоянна: размер холста — `192 px` на мировую единицу, число ячеек — примерно две на единицу. Для узких (1,02 × 1,63) и широких (1,95 × 1,25) панелей создаются отдельные наборы из четырёх вариантов. Палитра из шести цветов; каждый треугольник ячейки получает цвет по формуле от координат, варианта и индекса. Дополнительно рисуется круглое окно-роза из 12 секторов. Текстуры помечаются `SRGBColorSpace` и анизотропией 4.

14.2. Луч из панели

Для каждой панели `paneBeam` вычисляет:

- `origin` — точку ровно на внутренней грани стекла;
- `direction` — вектор к оси башни с наклоном вниз;
- `length` — расстояние до оси по горизонтали;
- `corners` — четыре мировые вершины грани;
- `sourceUV` — UV, скопированные именно с внутренней грани, а не с произвольной стороны.

Луч рисуется как четырёхгранная призма из вершин панели к «ковру» на полу. Шейдер луча добавляет мягкие края по UV и затухание к дальнему концу, включено аддитивное смешивание.

14.3. Проекция витража на полку

Каждое окно один раз выбирает полку: ближайшую по углу и самую высокую из тех, что ниже окна. Полки, зарезервированные под угрозы, пропускаются. На выбранной полке создаётся плоский декаль-

полигон; UV берутся с реальной стеклянной грани, поэтому рисунок сохраняет пропорции и количество элементов мозаики.

Фрагментный шейдер декаля:

- отбрасывает пиксели вне кольца полки и вне её углового сектора (`discard`);
- сравнивает глубину декаля с глубиной сцены из прохода `TowerMist` и гасит фрагменты, которые находятся не на поверхности — так проекция не «протекает» сквозь бортик и книги;
- рисует рамку, ромбовидную свинцовую решётку и звёздочку в центре;
- слегка пульсирует во времени.

14.4. Игровая зона

Для каждой проекции регистрируется зона с центром, касательным и радиальным векторами, шириной и длиной. `sampleLight(position, jumpHeight)` возвращает зону, если игрок находится в её прямоугольнике с допуском ±0,36 по высоте. При прыжке выше 3,5 единицы свет не действует — витраж лечит только стоящего на полке.

Зоны постоянны: они не исчезают после прохода, как и сказано в README. Палитра зоны берётся из материала панели и используется системой `VitralGlow` для цветного свечения зайчика.

15. Теневые звери

Четыре вида — кошка, волк, лиса и заяц — собираются из общих сфер и конусов. Материалы `MeshBasicMaterial` чёрного цвета с отключённым туманом: тень остаётся абсолютно чёрной и в закатном свете, и рядом со светящимся зайчиком. Глаза — два красных эллипсоида с ярким ядром без tone mapping.

Параметр	Значение
Расстановка	`s > 12`, шаг 28, смещение 16; виды чередуются
Преследование	При `|s − b.s| < 3` поперечная координата плавно тянется к игроку
Прыжок	`sin(time·2,7 + b.s)`, до 0,8 единицы
Урон	`|s−b.s| < 0,75`, `|lane−b.lane| < 1`, `height < 1,2`
Уязвимость	Прыжок сверху: `|s−b.s| < 0,58`, `|lane−b.lane| < 0,78`
Высота верха	1,02–1,15 в зависимости от вида

15.1. Смерть и растворение

При прыжке сверху зверь переводится в состояние `dead`. Все меши получают клонированный материал с шейдерной вставкой: фрагмент отбрасывается, если мировая высота ниже `dissolveHeight` плюс зернистый шум. Порог поднимается со скоростью 3,1 единицы в секунду, поэтому зверь «осыпается» снизу вверх. Одновременно 18 заранее созданных спрайтов дыма поднимаются вертикальными колоннами, зафиксированными в точке смерти. Через 1,7 с группа скрывается.

Пул спрайтов создаётся один раз при рождении зверя и переиспользуется: в кадре не происходит выделения памяти и создания материалов.

16. Визуальные эффекты

16.1. Сбор блика (`PickupEffects`)

Две фазы: сжатие 0,58 с с кубическим ease-in и вспышка 0,95 с. Во второй фазе расходятся 28 искр (`InstancedMesh`), кольцо, аура и точечный свет. По завершении все материалы, геометрия искр и группа удаляются из сцены — утечки исключены, что проверяет `pickup-effects.test.js`. `clear()` вызывается при перезапуске.

16.2. Теневой урон (`ShadowEffects`)

24 спрайта вращаются вокруг игрока по спирали. `setDamage(true)` запускает импульс только при входе в тень; при длительном уроне действует мягкое «сердцебиение», а не повторный удар. Экранное затемнение — обычный DOM-слой `#shadow-vfx`, прозрачность которого пишется из игры. Тряска камеры добавляется после `lookAt`. При `prefers-reduced-motion` тряска отключается, а пульсация становится статичной.

16.3. Тени на полу и дым (`ground-shadows.js`)

Пятно тени — шейдерный диск с волнистым краем, анимированным по времени. Дым — один `Points` с атрибутами `phase`, `life`, `moteSize`, `rise`: позиции и размеры вычисляются в вершинном шейдере, поэтому CPU в кадре не участвует. Размер точки масштабируется по перспективе и высоте буфера.

16.4. Свечение витража (`VitralGlow`)

12 конусов-лучей и 12 спрайтов равномерно распределены по сфере вокруг зайчика, плюс два точечных источника. Цвет берётся из палитры текущей зоны и плавно перетекает между оттенками. Сила свечения — `smoothstep` от оставшегося времени: при выходе из витража оно гаснет ровно за одну секунду. При нулевом свете группа невидима: погасший зайчик не может светить.

16.5. Туман (`TowerMist`)

96 инстансов одной плоскости образуют «объёмные» клубы. Вершинный шейдер покачивает их и разворачивает к камере; фрагментный строит FBM-шум Перлина, добавляет вертикальные карманы плотности, градиент от холодного низа к тёплому верху и ночной режим. Для контакта с геометрией каждый кадр выполняется отдельный проход глубины: сцена рендерится с `MeshDepthMaterial` в маленькую цель, прозрачные объекты и спрайты на это время скрываются. Разница глубин даёт мягкое прижатие тумана к полкам, а не «сквозь» них.

16.6. Анимация зайчика

`RabbitAnimation` двигает только вложенную группу `rabbitVisual`, не трогая корневую позицию — поэтому физика и анимация независимы. Три параметра смешиваются демпфированием: `blend` (скорость ходьбы), `air` (полёт) и `landing` (короткое приседание). Наклон корпуса зависит от

вертикальной скорости, крен — от поворота, сжатие сохраняет объём (`width = 1/√scaleY`). Уши, лапы и хвост получают процедурные фазы от общего `phase`.

17. Камера

Камера не следует за игроком непрерывно — она держит в кадре целую дугу спирали. Якорь `cameraAnchor` квантуется шагом, зависящим от соотношения сторон:

```
step = clamp(aspect · 1,35; 0,65; 2,2)    margin = 0,8 · step
```

Гистерезис (`while` по обе стороны от якоря) не даёт камере «дрожать» при развороте. Затем якорь сглаживается критически задемпфированной пружиной:

$$\text{offset} = \text{value} - \text{target} \qquad \text{carry} = \text{velocity} + 7 \cdot \text{offset}$$

$$\text{value}' = \text{target} + (\text{offset} + \text{carry} \cdot dt) \cdot e^{-7dt}$$

$$\text{velocity}' = (\text{velocity} - 7 \cdot \text{carry} \cdot dt) \cdot e^{-7dt}$$

Такая схема сохраняет непрерывность скорости при выборе новой дуги. Поза камеры ставится на 0,45 рад позади фокуса, поднимается на 7,6 единицы и масштабируется коэффициентом `zoom = 0,92 · max(1; 1,45/aspect)`, чтобы на узких экранах дуга не обрезалась.

Тест `game.test.js` прогоняет весь маршрут в обе стороны при трёх соотношениях сторон, с крайними значениями `lane` и в прыжке, и требует, чтобы проекция зайчика оставалась в пределах 85 % кадра. В интро и финале используются отдельные плавные траектории.

18. Освещение и шейдеры

Источник	Параметры	Роль
HemisphereLight	#a1bdcc / #5a3849, 0,16…2,1	Общая заполняющая засветка
DirectionalLight	#ffcf87, 0,1…4,1	Закатное солнце снаружи
PointLight «тёплый»	#ffa769, 4…140	Свет в сердце башни
PointLight кристалла	#a2ffeb, 20 → 250 в финале	Финальное сияние
Свет блика / витража	локальные, короткоживущие	Вспышки и цветные отсветы

Интенсивности интерполируются от `depthLight(s)`: сверху мир залит тёплым светом, внизу остаются холодные сумерки. Фон и туман интерполируются между `#303849` и `#080c17`, плотность тумана растёт с глубиной.

18.1. Затемнение архитектуры

`applyTowerDepth` не заменяет материал, а внедряется в его шейдер через `onBeforeCompile`. В вершинной части вычисляется мировая высота (с учётом инстансинга), во фрагментной — коэффициент `towerDepth`

из двух факторов: абсолютной высоты и того, насколько объект ниже камеры. Итоговый цвет смешивается с тёмно-синим и слегка приглушается. `customProgramCacheKey` фиксирует версию программы, чтобы Three.js не собирал шейдер повторно.

18.2. Тени

Карты теней реального времени не используются. «Тени» в игре — это игровые зоны (`hazards`), шейдерные пятна на полу, чёрные модели зверей и дым. Такой подход дешевле, полностью управляем и не зависит от разрешения карты теней. Это осознанное решение, а не упрощение рендера.

18.3. Тон-маппинг

Включён `ACESFilmicToneMapping` с экспозицией 1,25; соотношение пикселей ограничено `min(devicePixelRatio, 1,7)`. Шейдеры лучей, декалей и тумана вручную включают фрагменты `tonemapping_fragment` и `colorspace_fragment`, чтобы совпадать с остальной сценой.

19. Интерфейс

Интерфейс — обычный DOM поверх canvas, без фреймворка и виртуального DOM. Обновляются только несколько узлов за кадр:

- `#light-value`, `#light-fill` — свет и его полоса;
- `#progress` — прогресс по маршруту;
- `#floor` — номер этажа, `#shards` — счётчик бликов;
- `#effect` — текстовое состояние («Тень забирает свет!», «Витраж · восстановление ×2»);
- `#toast` — подсказки с `role="status"` и `aria-live="polite"`.

Панели `#intro`, `#hud` и `#modal` переключаются атрибутом `hidden` и классом `body.playing`. Модальное окно одно и переиспользуется для паузы, победы и поражения: меняются только заголовок, текст и подпись кнопки. Сенсорные кнопки находятся в `#touch` и активируются через `data-key` — тот же код, что и клавиатура.

Доступность: у canvas есть `aria-label`, у кнопок — подписи, видимый фокус задан через `:focus-visible`, а анимационные эффекты учитывают `prefers-reduced-motion`.

20. Звук

Звук синтезируется на лету: осциллятор синусоиды, короткая огибающая усиления, частоты от 164 до 1100 Гц. `AudioContext` создаётся лениво при первом включении звука — это требование браузеров к пользовательскому жесту. По умолчанию звук выключен, кнопка переключает состояние и подпись. Отдельных аудиофайлов нет, поэтому сборка остаётся полностью статической.

21. Производительность

- **Инстансинг.** Книги декора — 6 вызовов отрисовки; туман — 1 вызов на 96 плоскостей; искры блика — 1 вызов на 28 частиц.
- **Пулы.** Дым зверей (18 спрайтов) и искры создаются заранее и переиспользуются.
- **Ограничение разрешения.** `pixelRatio ≤ 1,7`; на мобильных это заметно снижает нагрузку на фрагментные шейдеры.
- **Один проход глубины.** Он нужен и туману, и декалям витражей; прозрачные объекты на время прохода скрываются, поэтому стоимость ограничена непрозрачной геометрией.
- **Отсечение.** Геометрия сцены статична и проходит обычное фрустум-отсечение; у частиц отсечение отключено осознанно, чтобы не пропадали эффекты.
- **Отсутствие физического движка.** Все коллизии — сравнения скаляров, без raycast и без broadphase.

> **Осознанный компромисс.** `route()` возвращает новый объект `{x, y, z, theta}` при каждом вызове, а `update()` вызывает её десятки раз за кадр. Это создаёт короткоживущий мусор. При желании функцию можно переписать на запись в переданный вектор; сейчас читаемость и тестируемость признаны важнее. Аналогично `new THREE.Vector3` в кадре встречается в местах, где важна ясность кода.

22. Тестирование

Используется встроенный `node --test`. Тесты не требуют браузера и WebGL: проверяются чистые функции и логика, а для модулей с Three.js создаются объекты сцены в Node (Three.js работает без рендерера). Всего 28 тестов в 13 файлах.

Файл	Что проверяется
`game.test.js`	Поворот по кратчайшей дуге; гистерезис камеры; зайчик в кадре на всём маршруте при трёх aspect; непрерывный спуск; разрывы; формула света
`jump.test.js`	Геометрия разрыва; перелёт всех разрывов на 25–144 FPS в обе стороны; падение при ходьбе в дыру
`progression.test.js`	Ровно 12 этажей и монотонное угасание; условия прыжка на зверя
`hp-color.test.js`	Яркость 1 при 100 HP, 0 при нуле, резкое падение ниже половины, монотонность
`shelf-length.test.js`	Разброс длины полок в пределах ⅛ и сохранение проходимой полосы
`threats.test.js`	Состав реестра, резервирование, цикл полки, потеря опоры на внешнем крае
`window-light.test.js`	Луч начинается на внутренней грани; все стороны светят внутрь; проекция на полке; глубина; резерв
`stained-glass.test.js`	Плотность текселей; каждая панель проецируется один раз с полной мозаикой и пропорциями
`tower-mist.test.js`	Проход глубины скрывает VFX, восстанавливает состояние и цель рендера даже при ошибке
`pickup-effects.test.js`	Фазы эффекта и полное освобождение объектов
`shadow-effects.test.js`	Импульс только при входе в тень; затухание; reduced motion
`rabbit-animation.test.js`	Анимация не двигает корень; затухание; приседание; сброс
`vitral-glow.test.js`	Палитра зоны; исчезновение ровно через секунду; нулевой свет

Запуск: `npm test`. Тесты входят в CI перед сборкой, поэтому сломанная математика не попадёт на GitHub Pages.

23. Сборка и развёртывание

Команда	Действие
`npm ci`	Установка зависимостей по lock-файлу
`npm run dev`	Vite на `127.0.0.1` с горячей перезагрузкой
`npm run build`	Статическая сборка в `dist/`
`npm run preview`	Проверка production-сборки
`npm test`	28 тестов на `node:test`

`vite.config.js` задаёт `base: './'`, поэтому сборка работает и в корне домена, и по адресу `username.github.io/repository/`. Workflow `deploy.yml` на push в `main` или ручной запуск выполняет `npm ci` → `npm test` → `npm run build` → публикацию артефакта `dist` через `actions/deploy-pages`. Открытие `index.html` по `file://` не работает из-за ES-модулей — нужен HTTP-сервер.

24. Точки расширения

Задача	Что менять
Добавить разрыв	Дописать секцию в `gaps`; пересчитать резервы и проверить тесты прыжка
Добавить тип угрозы	Расширить `ThreatType`, `createThreats`, `unsupportedThreat`; создать визуал в `main.js`
Изменить баланс	Константы `PASSIVE_LIGHT_DRAIN`, `VITRAL_LIGHT_REGEN`, `JUMP_SPEED`, `GRAVITY` в `game.js`
Добавить предмет	Новый массив-запись, условие расстановки в цикле секций, обработка в `update`, эффект по образцу `PickupEffects`
Добавить вид зверя	Ветка в `createShadowBeast`, значение в `BEAST_KINDS`, при необходимости `stompTop`
Увеличить число этажей	`FLOOR_COUNT` и формулу `floorAt` (14 секций на этаж), `END`, высоты ярусов и окон; обновить тесты
Настроить графику	`depthLight`, интенсивности источников, `applyTowerDepth`, uniform'ы тумана

> **Осторожно.** Формулы расстановки объектов завязаны на индексы секций (`s % 5` , `s % 11` , `s % 27` , `s % 28` , `s % 29`) и на множество зарезервированных полок. При изменении `END` или шага спирали нужно синхронно пересмотреть и расстановку, и резервы, иначе предметы начнут появляться в разрывах или на подвижных полках.

25. Расхождения и технический долг

25.1. README и код

> **Расхождение в числах света.** README указывает пассивную потерю 0,96 HP/с и восстановление в витраже 1,92 HP/с. В коде действуют значения `PASSIVE_LIGHT_DRAIN = 12` и `VITRAL_LIGHT_REGEN = 24` , то есть 12 и 24 единицы в секунду. Документ описывает код; README в этой части устарел.

Остальные числа README совпадают с реализацией: 12 этажей по 14 секций, 168 секций, потеря 14 при провале, +22 за блик, уход полки на 2,1 единицы каждые 5 секунд, отсутствие иммунитета в витраже, четыре вида зверей.

25.2. Неиспользуемый экспорт

`fellIntoGap(s, height, velocity)` экспортируется и покрыт тестами, но в игровом цикле не вызывается: проверка опоры выполняется через `unsupportedThreat` , который учитывает и разрывы, и подвижные полки. Функцию можно либо использовать в `update` , либо удалить вместе с тестом.

25.3. Смешение ответственности в `main.js`

Модуль композиции одновременно содержит построение сцены, расстановку сущностей, состояние, ввод, HUD и рендер. Для проекта такого размера это приемлемо, но при дальнейшем росте естественные шаги — вынести `update(dt)` в отдельный модуль симуляции и передавать в него ссылки на сущности, оставив в `main.js` только создание сцены и кадр.

25.4. Аллокации в кадре

Как отмечено в разделе 21, `route()` и несколько вызовов `new THREE.Vector3` создают короткоживущие объекты. При необходимости оптимизации это первое место для замены на переиспользуемые векторы и запись по ссылке.

Документ подготовлен по исходному коду проекта «Солнечный зайчик — Башня последнего света» (каталог `Sunny`). Все приведённые значения соответствуют версии 1.0.0.